Benchmarking BraTS Baseline Using nnU-Net

Yifan HE, Shandong University

September 26, 2024

Abstract

In this experiment, I used the nnU-Net v2 model and investigated its powerful image segmentation capabilities. Meanwhile, learning about the domain of medical image processing, which is one of my interests.

1 Brief Introduction

1.1 Question 1

Make a brief explanation of the BraTS dataset, highlighting its significance in brain tumor segmentation.

BraTS is a medical imaging dataset which is dedicated to brain tumor segmentation, usually consists of multimodal MRI scans. The aim of this dataset is to provide accurate brain tumor segmentation annotation, especially to distinguish different subregions of the tumor, such as enhancing tumor, tumor core, edema area, and whole tumor region. It has important application value in medical image analysis, and has become an important benchmark in this field.

1.2 Question 2

Provide an overview of nnU-Net, focusing on its adaptability and performance in medical image segmentation tasks.

nnU-Net is a medical image segmentation framework which can adapt to **any new dataset**, analyze the traits of the given dataset to automatically adjust all hyperparameters without any artificial intervention. It did not make any improvement to the network but was able to achieve first place in many challenges simply by adapting to the dataset. As a result, nnU-Net has rapidly evolved and became a widely used benchmark in medical image segmentation.

2 Experiment

Attention! Since nnU-Net has extremely strict requirements for dataset formats, file paths, file naming, etc., users must follow the official tutorials carefully, otherwise, some complex errors may occur while using this model.

CPU: 6 vCPU Intel(R) Xeon(R) Gold 6130 CPU @ 2.10GHz, GPU: Tesla V100-PCIE-32GB * 1

2.1 Data Processing

Please make sure that nnU-Net has been installed correctly and that the dataset has been obtained. If data files are in **DICOM** format, please convert to **NIfTI** format (Use **dicom2nifti** for conversion). Create three folders, **nnUNet_raw** for the original dataset, **nnUNet_preprocessed** for the preprocessed dataset, and **nnUNet_results** for the results. The internal format of datasets must comply with the requirement! (Reference: Link.)

For MSD type files, conversion is necessary. The ID generated during the process is customized and important, serving as a pointer for the program for recognition.

I used 100 samples for the training set (with a small portion selected as the validation set) and 20 samples for the test set. I did not significantly reduce the sample size to ensure the model learns enough information while also improving training speed. Additionally, the test set helps assess the model's reliability and the validity of the evaluation metrics.

2.2 Train

The images in dataset cannot be processed as 3D_lowres, so only 2D and 3D_fullres can be obtained after preprocessing. Program supports 5-fold cross-validation. To achieve a more comprehensive evaluation, make full use of data, and obtain the best model, 5 training runs (1 for each fold) should be performed for both 2D and 3D_fullres. Otherwise, the evaluation may become biased. However, as many previous experiments have shown that 3D_fullres usually has a better performance than 2D, I conducted 5 training runs (1 for each fold) only on 3D_fullres. This also saves time. For detailed issues during training (e.g. epoch, batch_size), please read the `readme.md` file.

For the dataset size, epoch, and other details I set, it takes about 0.5 minute to train 1 epoch, and about 2.5 hours to completely train once. Therefore, 5 training sessions take about 12.5 hours. Since the rented server doesn't support running code after shutdown, I actually spent nearly 3 days training the model, during which I encountered many problems.

2.3 Infer

Before inferring, it is necessary to determine the best nnU-Net configuration. Furthermore, prepare the test set (imageTs) and a folder for storing inference results. nnUNet provides post-processing function, in order to remove noise and improve the quality of results, I strongly recommend to perform it.

3 Problems

Here is a summary of the troubles I encountered.

1. Unable to recognize commands. This is a path that has not been fully set. If manually set the path will be more complicated, so I recommend to read `pyproject.toml` file to check the location of a certain command, find corresponding file, and manually add the recognition path using `sys` package.

2. `RuntimeError:unable to write to file </torch_86313_780457312_0>: No space left on device(28).` The obvious reason is that there is a problem with the Dorker space, but there is no good solution. I will provide a solution in the following problem.

3. Although the path has been set, the program did not process the prepared dataset, but downloaded some non-existent DICOM files. In my opinion, nnU-Net was contaminated for unknown reasons during installation. The solution is to uninstall it, create a new conda environment, re-download and configure all necessary content.

4. `RuntimeError:One or more background workers are no longer alive.Exiting.` This is the worst issue, involving a process problem, and it wasted almost an entire day of my time. After my detailed examination, the process became **False** in less than 1 second of running and did not even enter the GPU. No matter how I adjusted the number of processes or
followed the official method, the issue couldn't be solved. Therefore, I switched to another server (the original server is configured with **Nvidia GeForce RTX 3090** $\times$ **4**).

4 Results

In this chapter, I will present some important metrics.

The following table shows 2 important metrics officially used for final evaluation. The remains (sensitivity, specificity, etc.) are for reference only and are not considered as ranking requirements.

Metrics		Private	Baseline	Rank 1
Dice	Dice(WT)	0.9197	0.8895	0.8925
	Dice(TC)	0.8875	0.8506	0.8669
	Dice(ET)	0.8463	0.8203	0.7890
HD95	HD95(WT)	3.50869	8.498	4.582576
	HD95(TC)	3.05029	17.337	4.898979
	HD95(ET)	3.04712	17.805	3

Table 1: Official evaluation metrics (Dice Score & Hausdorff Distance 95)

The table below shows 2 metrics recommended by BraTS Official as references.

Metrics		Private	Rank 1
Sensitivity	Sensitivity(WT)	0.914042	0.932178
	Sensitivity(TC)	0.879024	0.8506
	Sensitivity(ET)	0.856740	0.8203
Specificity	Specificity(WT)	0.999408	0.99888
	Specificity(TC)	0.999795	0.999891
	Specificity(ET)	0.999662	0.99989

Table 2: Reference metrics (Sensitivity & Specificity)

In addition, I've also added other metrics.

Metrics		Region-based
HD	HD(WT)	3.8179
	HD(TC)	3.3185995
	HD(ET)	3.28150437
IoU	IoU(WT)	0.85858475
	IoU(TC)	0.82081
	IoU(ET)	0.750376

Table 3: Other metrics (Hausdorff Distance & IoU)

I also run category-based training. Here are some metrics I obtained.

Metrics		Category-based
Dice	edema	0.773851
	non-ET	0.558308
	ET	0.699536
IoU	edema	0.6485758
	non-ET	0.420422
	ET	0.599614
Sensitivity	edema	0.79109
	non-ET	0.57406
	ET	0.74810
Specificity	edema	0.99856
	non-ET	0.99935
	ET	0.99975
HD	edema	4.505706
	non-ET	4.24975
	ET	3.26598
HD95	edema	4.19234
	non-ET	3.958102
	ET	3.048204

Table 4: Category-based training

Here are some significant equations.

$$Sensitivity = \frac{TP}{TP + FN}$$

$$Specificity = \frac{TN}{TN + FP}$$

$$IoU = \frac{A \cap B}{A \cup B} = \frac{TP}{TP + FP + FN}$$

$$Dice = \frac{2 \times |A \cap B|}{|A| + |B|} = \frac{2 \times TP}{(TP + FN) + (TP + FP)}$$

$$HD = \max \left\{ \max_{a \in A} \min_{b \in B} ||a - b||, \max_{b \in B} \min_{a \in A} ||b - a|| \right\}$$

Where **TP** means True Positive, **FP** means False Positive, **TN** means True Negative and **FN** means False Negative. Besides, **A** is the Ground-Truth Region and **B** is the Segmentation Region(Predicted). Thus, it can be concluded that:

$$TP = |A \cap B|$$

$$FP = |B| - |A \cap B|$$

$$TN = |U| - |A \cup B|$$

$$FN = |A| - |A \cap B|$$

5 Conclusions

For this project, I used the extremely powerful model nnUNetv2 in the field of medical image processing, analyzed MRI medical images to accurately confirm the location of tumors and different tumor regions as much as possible. According to the results I obtained, it is obvious that for region-based training, the model performs very well, overtaking the official baseline and even being comparable to the performance of the Rank 1 model. However, the amount

of the data I used for training is less than that used in official baseline, which may be the reason why my final results surpassed the baseline. Besides, I found a confusing phenomenon that nnU-Net performs better when conducting region-based training. The reason may be that when the model conducts region-based training, the labels are organized into higher-level semantic aggregates(categories are combined in different ways), allowing the model to treat different categories as a whole. This method may reduce interference between different categories, and at the same time, the model may be able to more effectively capture tumor features. For potential improvements, the baseline nnU-Net kernel is a 3D U-Net that runs on patches of size $128 \times 128 \times 128$. This network has an Encoder-Decoder structure with skip connections which link 2 paths together. Therefore, I ponder the most fundamental way to improve nnU-Net is to increase the network size. For instance, asymmetrically enlarged network structures, increase the number of kernels in encoder, while maintaining the same number of kernels in decoder, while ensuring that the output of the encoder can be accepted by the decoder(Make sure that the shape of the encoder output can be received by decoder). What's more, I want to import the attention mechanism if possible. In addition, modifying the training strategy may improve the model and achieve better results, such as the update method of the learning rate, etc..The official learning rate update formula is as follows:

$$new\_lr = initial\_lr \times \left(1 - \frac{current\_epoch}{max\_epoch}\right)^{exponent}$$

Perhaps utilizing a learning rate scheduler to cyclically vary the learning rate between two dynamically adjustable boundaries could lead to unexpected improvements.

For details, please read `README.md` document.Thanks!